Royaume du Maroc

Ministère de l'Enseignement Supérieur, de la Recherche Scientifique et de l'Innovation

Université Ibn Zohr

Ecole Nationale Supérieure de l'Intelligence Artificielle et Sciences des Données

Rapport de Stage

Spécialité : Science des Données, Big Data et Intelligence Artificielle

Présenté par

Aglagal Khadija

Détection d'intrusions par apprentissage automatique (jeux de données publics)

Devant le jury

Pr	Monsieur AZIZI Mohamed Saad	Encadrant

Année Universitaire 2026 - 2027

Dédicaces

Je dédie ce travail :

- À **Monsieur AZIZI Mohamed Saad**, mon encadrant, pour son accompagnement, ses conseils précieux, sa disponibilité et ses encouragements tout au long de la réalisation de ce projet.
- À **mes parents**, pour leur amour, leur soutien inconditionnel, leurs sacrifices et leurs encouragements qui m'ont permis d'avancer et de persévérer dans mes études.
- À **ma famille**, pour leur présence, leur affection et leur soutien durant les moments difficiles comme dans les moments de réussite.
- À **mes enseignants**, pour leurs efforts, leur patience et les connaissances qu'ils m'ont transmises tout au long de mon parcours académique.
- À **mes amis et mes collègues**, pour leur soutien, leur aide et les beaux moments partagés durant cette période.
- Enfin, à **toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce travail**, je leur adresse mes sincères remerciements et ma profonde gratitude.

À vous tous, je dédie ce travail avec toute ma reconnaissance.

Remerciements

Avant toute chose, je tiens à exprimer ma profonde gratitude à **Dieu Tout-Puissant** pour m'avoir donné la force, la patience et la persévérance nécessaires à la réalisation de ce travail. Je souhaite adresser mes sincères remerciements à l'ensemble du **cadre administratif et pédagogique de l'ENSIASD**, pour leur accueil, leur accompagnement et les efforts déployés afin de nous offrir un environnement d'apprentissage favorable et de qualité. Je remercie également **tous les enseignants et professeurs de l'ENSIASD** pour leur disponibilité, leur patience, leurs conseils et pour les connaissances et compétences qu'ils nous ont transmises tout au long de notre formation. Leur enseignement et leur accompagnement ont largement contribué à enrichir mon parcours académique et à développer mes compétences. J'adresse une reconnaissance particulière à **Monsieur AZIZI Mohamed Saad**, mon encadrant, pour son accompagnement, sa disponibilité, ses conseils pertinents et ses orientations tout au long de la réalisation de ce projet. Ses remarques et ses recommandations m'ont permis d'améliorer mon travail et de mieux comprendre les différentes étapes de sa réalisation. Je tiens également à remercier **toutes les personnes qui ont contribué, directement ou indirectement, à la réalisation de ce projet**, pour leur aide, leur soutien et leurs encouragements. Enfin, je remercie chaleureusement **ma famille et mes proches** pour leur soutien moral, leur patience et leurs encouragements qui m'ont permis de poursuivre mes efforts et d'aller jusqu'au bout de ce travail. À toutes ces personnes, j'adresse mes **sincères remerciements et ma profonde reconnaissance**.

Table des matières

Introduction générale **1**

1 Contexte général **2**

1.1 Étude et critique de l'éxistant . . . 3

1.1.1 Les méthodes traditionnelles de détection . . . 3

1.1.2 Les systèmes de détection d'intrusions . . . 3

1.1.3 Étude bibliographique . . . 4

1.2 Problématique . . . 4

1.3 Les menaces et les enjeux de la cybersecurite . . . 4

1.3.1 L'apport du Machine Learning dans la cybersécurité . . . 5

1.3.2 Définition et classification des IDS . . . 6

1.4 Solution proposée et objectifs globaux du projet . . . 7

2 Analyse et spécification des besoins **9**

2.1 Identification des acteurs du système . . . 10

2.2 Identification des besoins . . . 10

2.2.1 Besoins fonctionnels . . . 11

2.2.2 Besoins non fonctionnels . . . 11

2.2.3 Diagramme de cas d'utilisation . . . 12

2.2.4 Diagramme de séquence . . . 13

2.3 Diagramme de GANTT théorique . . . 13

3 Conception et choix technologiques **15**

3.1 Conception architecturale du système . . . 16

3.1.1 Architecture générale . . . 16

3.1.2 Présentation du dataset . . . 17

3.1.3 Problèmes dans les Données . . . 20

3.2 Préparation et prétraitement des données . . . 20

3.2.1 Nettoyage des données . . . 20

3.3 Principes du Machine Learning pour la Détection d'Intrusions . . . 23

3.3.1 Les Algorithmes de Classification Utilisés . . . 24

3.4 Techniques d'Évaluation des Modèles . . . 24

3.4.1 Accuracy, Precision, Recall et F1-Score . . . 24

3.5 Choix du Modèle Optimal 26

4 Réalisation **27**

4.1 Choix techniques 28

4.1.1 Flask 28

4.1.2 sqlite 28

4.1.3 HTML 28

4.1.4 CSS 29

4.1.5 Bootstrap 29

4.1.6 Javascript 29

4.2 l'environnement de développement 30

4.3 Travail réalisé 31

4.3.1 Page d'accueil 31

4.3.2 Section "About" 31

4.3.3 Pages d'authentification 32

4.3.4 Interface de prédiction 32

4.3.5 Résultat de la prédiction 34

Conclusion générale **35**

Bibliographie **37**

Annexes **38**

Table des figures

2.1 Diagramme de cas d'utilisation du système 12
2.2 Diagramme de séquence de la détection d'intrusion 13
2.3 Diagramme de Gantt du projet 14

3.1 Architecture générale du système de détection d'intrusions 16
3.2 Répartition des classes dans le dataset 19
3.3 Heatmap des valeurs manquantes dans le dataset 21
3.4 État des valeurs manquantes après le nettoyage 21
3.5 Top 10 Features les plus importantes 23

4.1 Flask 28
4.2 sqlite 28
4.3 HTML 29
4.4 CSS 29
4.5 Bootstrap 29
4.6 Javascript 30
4.7 Vs code 30
4.8 Jupyter Notebook 30
4.9 latex 31
4.10 Page d'accueil de l'application 31
4.11 Section "About" de l'application 32
4.12 Pages d'inscription et de connexion 32
4.13 Interfaces de prédiction de l'application 33
4.14 Résultat de la prédiction après import CSV 34

Liste des tableaux

3.1 Exemples de caractéristiques du dataset 18
3.2 Répartition des classes dans le dataset 19
3.3 Comparaison des algorithmes de classification utilisés 24
3.4 Comparaison des performances des modèles 26

4.1 Liste des 23 caractéristiques utilisées pour l'entraînement 38

Liste des abréviations

IDS *(Intrusion Detection System)* – Système de détection d'intrusion.

ML *(Machine Learning)* – Apprentissage automatique.

IA Intelligence Artificielle.

RF *(Random Forest)* – Forêt aléatoire.

DT *(Decision Tree)* – Arbre de décision.

TCP *(Transmission Control Protocol)* – Protocole de contrôle de transmission.

UDP *(User Datagram Protocol)* – Protocole de datagramme utilisateur.

HTTP *(HyperText Transfer Protocol)* – Protocole de transfert hypertexte.

IP *(Internet Protocol)* – Protocole Internet.

DDoS *(Distributed Denial of Service)* – Attaque par déni de service distribué.

CSV *(Comma-Separated Values)* – Fichier de valeurs séparées par des virgules.

CSS *(Cascading Style Sheets)* – Feuilles de style en cascade.

HTML *(HyperText Markup Language)* – Langage de balisage hypertexte.

SQL *(Structured Query Language)* – Langage de requête structuré.

UML *(Unified Modeling Language)* – Langage de modélisation unifié.

API *(Application Programming Interface)* – Interface de programmation applicative.

UI *(User Interface)* – Interface utilisateur.

Introduction générale

À l'ère de la transformation numérique, les technologies de l'information et les réseaux informatiques occupent une place essentielle dans notre quotidien. Ils permettent de faciliter la communication, le partage des informations et l'accès à de nombreux services, aussi bien dans les entreprises que dans les établissements publics et privés.

Cependant, cette évolution technologique s'accompagne d'une augmentation constante des risques liés à la sécurité informatique. Les systèmes et les réseaux sont régulièrement exposés à différentes menaces pouvant compromettre la confidentialité, l'intégrité et la disponibilité des données et des services. Les cyberattaques représentent ainsi un enjeu majeur pour les organisations, quelle que soit leur taille ou leur domaine d'activité.

Face à ces menaces, la cybersécurité est devenue une nécessité afin de protéger les ressources informatiques et de garantir un fonctionnement fiable et sécurisé des systèmes. La surveillance des réseaux, l'identification des comportements anormaux et la détection des activités malveillantes constituent notamment des éléments importants dans la mise en place d'une stratégie de sécurité efficace.

Par ailleurs, l'évolution des technologies et des méthodes d'analyse offre aujourd'hui de nouvelles possibilités pour renforcer la sécurité des systèmes informatiques. L'intelligence artificielle et l'apprentissage automatique suscitent notamment un intérêt croissant dans différents domaines de la cybersécurité.

Ainsi, la protection des réseaux et des systèmes d'information constitue aujourd'hui un défi permanent, nécessitant des solutions capables de s'adapter à l'évolution des menaces et aux nouvelles formes de cyberattaques.

Chapitre 1

Contexte général

Plan

1.1 Étude et critique de l'éxistant 3

1.2 Problématique 4

1.3 Les menaces et les enjeux de la cybersecurite 4

1.4 Solution proposée et objectifs globaux du projet 7

Introduction

La transformation numérique et le développement des technologies de l'information ont profondément modifié la manière dont les données et les services sont utilisés et échangés. Les réseaux informatiques sont aujourd'hui indispensables au fonctionnement de nombreuses organisations et jouent un rôle central dans la communication et l'accès aux ressources numériques.

Cette évolution s'accompagne toutefois d'une augmentation des risques liés aux cybermenaces. Les systèmes informatiques peuvent être exposés à différentes formes d'activités malveillantes susceptibles d'affecter la sécurité des données et la disponibilité des services. La protection des infrastructures informatiques constitue donc un enjeu majeur pour les organisations.

Dans ce contexte, la détection des activités suspectes et la prévention des intrusions représentent des problématiques importantes dans le domaine de la cybersécurité.

1.1 Étude et critique de l'éxistant

1.1.1 Les méthodes traditionnelles de détection

Les premières approches de détection des intrusions reposent principalement sur l'utilisation de règles et de signatures correspondant à des comportements ou attaques déjà connus. Lorsqu'une activité observée correspond à une signature enregistrée, une alerte peut être générée.

Cette approche présente l'avantage d'être relativement simple à mettre en œuvre et efficace pour détecter des attaques connues. Cependant, elle présente certaines limites lorsqu'elle est confrontée à de nouvelles menaces ou à des comportements qui ne correspondent pas aux signatures existantes.

1.1.2 Les systèmes de détection d'intrusions

Les systèmes de détection d'intrusions, communément appelés IDS (*Intrusion Detection Systems*), permettent de surveiller les activités d'un système ou d'un réseau afin d'identifier des comportements susceptibles de représenter une menace.

On distingue généralement deux grandes approches :

- **La détection basée sur les signatures :** elle recherche des caractéristiques correspondant à des attaques déjà connues.
- **La détection basée sur les anomalies :** elle cherche à identifier des comportements qui s'écartent du comportement considéré comme normal.

Malgré leur utilité, ces approches peuvent rencontrer certaines difficultés face à la complexité croissante des réseaux et à l'évolution constante des cyberattaques.

1.1.3 Étude bibliographique

La détection des intrusions constitue un domaine de recherche largement étudié dans le contexte de la cybersécurité. De nombreux travaux scientifiques se sont intéressés à l'amélioration des mécanismes de détection et à l'utilisation de nouvelles méthodes d'analyse.

Les recherches récentes accordent notamment une importance particulière aux techniques d'intelligence artificielle et d'apprentissage automatique. Ces techniques permettent d'analyser de grandes quantités de données et d'identifier des relations ou des comportements difficiles à détecter uniquement à l'aide de règles prédéfinies.

L'apprentissage automatique peut être utilisé dans différents scénarios de cybersécurité, notamment pour la classification du trafic réseau, la détection d'anomalies et l'identification de comportements potentiellement malveillants [1].

Cependant, les performances des modèles dépendent fortement de plusieurs facteurs, notamment la qualité des données, le choix des caractéristiques, l'équilibre entre les différentes classes et la méthode d'évaluation utilisée. Il est donc nécessaire d'adopter une démarche rigoureuse lors de la conception et de l'évaluation de ces systèmes.

1.2 Problématique

L'augmentation du nombre et de la diversité des cyberattaques rend la sécurisation des réseaux de plus en plus complexe. Les systèmes informatiques doivent être capables de faire face à des comportements malveillants qui peuvent évoluer rapidement et prendre différentes formes.

La principale difficulté réside alors dans la capacité à distinguer efficacement les activités normales des activités potentiellement malveillantes, tout en limitant les erreurs de détection. Une détection insuffisante peut laisser passer certaines menaces, tandis qu'un nombre trop important de fausses alertes peut rendre la surveillance difficile et diminuer l'efficacité des équipes de sécurité.

La problématique générale peut ainsi être formulée comme suit :

Comment améliorer la détection des activités malveillantes au sein d'un réseau informatique tout en garantissant une détection fiable et adaptée à l'évolution des cybermenaces ?

1.3 Les menaces et les enjeux de la cybersecurite

Les cyberattaques sont de plus en plus diversifiées et peuvent prendre plusieurs formes, notamment :

- **Les attaques par phishing** : tentatives de vol d'informations sensibles par des e-mails frauduleux ou des sites web usurpés.

- **Les attaques DDoS** : surcharge massive d'un serveur par des requêtes malveillantes, rendant un service indisponible.
- **Les logiciels malveillants** : virus, ransomwares et chevaux de Troie qui exploitent des vulnérabilités pour infecter les systèmes.
- **Les intrusions réseau** : accès non autorisé à des systèmes informatiques, pouvant conduire à une exfiltration de données sensibles.
- **Les attaques adversariales contre l'IA** : manipulation intentionnelle des modèles de Machine Learning à l'aide de données trompeuses.

Les conséquences de ces attaques sont désastreuses et peuvent engendrer :

- **Des pertes financières considérables** pour les entreprises (ex. : l'attaque WannaCry de 2017 a causé des milliards de dollars de pertes).
- **Une atteinte à la réputation** des organisations ciblées.
- **Des fuites de données sensibles**, compromettant la vie privée des utilisateurs.
- **Des risques systémiques** dans des secteurs critiques tels que l'énergie, la finance et la santé.

Ainsi, la mise en place de mécanismes avancés de protection est impérative afin d'anticiper ces menaces et de minimiser leur impact.

1.3.1 L'apport du Machine Learning dans la cybersécurité

L'intelligence artificielle et, plus particulièrement, le Machine Learning constituent aujourd'hui des technologies prometteuses dans le domaine de la cybersécurité. Elles permettent d'améliorer les méthodes traditionnelles de détection en analysant automatiquement de grandes quantités de données et en identifiant des comportements inhabituels ou potentiellement malveillants.

Contrairement aux approches classiques, le Machine Learning permet notamment :

- **Une analyse automatisée des données** : les modèles peuvent traiter rapidement de grandes quantités de données provenant du trafic réseau et des journaux d'événements.
- **Une détection efficace des anomalies** : l'apprentissage automatique permet d'identifier des comportements inhabituels pouvant correspondre à des tentatives d'intrusion.
- **Une amélioration de la précision** : les algorithmes de classification peuvent distinguer les activités normales des activités malveillantes tout en limitant les erreurs de détection.

- **Une détection proactive des menaces** : les modèles peuvent apprendre à partir de données historiques afin de reconnaître des comportements associés à différentes catégories d'attaques.
- **Une automatisation de certaines tâches de sécurité** : l'utilisation du Machine Learning permet de réduire l'intervention humaine dans certaines étapes de l'analyse et de la détection.

Cependant, l'utilisation du Machine Learning dans la cybersécurité présente également plusieurs défis :

- **Le choix de l'algorithme** : plusieurs méthodes de classification peuvent être utilisées, telles que les arbres de décision, Random Forest, KNN ou encore les réseaux de neurones. Le choix dépend notamment de la nature et des caractéristiques des données.
- **La qualité des données** : les performances du modèle dépendent fortement de la qualité, de la représentativité et de la préparation des données utilisées pour son apprentissage.
- **La gestion des faux positifs et des faux négatifs** : une mauvaise classification peut entraîner soit des alertes inutiles, soit l'absence de détection d'une véritable attaque.
- **La robustesse des modèles** : les systèmes de détection doivent être suffisamment robustes pour faire face à des comportements nouveaux ou différents de ceux observés durant l'entraînement.
- **Le coût computationnel** : l'entraînement et l'utilisation de certains modèles peuvent nécessiter des ressources matérielles importantes, notamment lorsque les volumes de données sont élevés.

Dans le cadre de ce projet, le Machine Learning est utilisé pour développer un système de détection d'intrusions capable d'analyser le trafic réseau et de distinguer les activités normales des activités malveillantes. L'approche adoptée repose sur un algorithme de classification entraîné à partir d'un jeu de données contenant différents comportements réseau. L'objectif est ainsi de construire une solution capable d'améliorer la détection automatique des intrusions et de contribuer au renforcement de la sécurité des systèmes informatiques.

1.3.2 Définition et classification des IDS

Un **IDS (Intrusion Detection System)** est un système de sécurité informatique conçu pour surveiller les activités d'un réseau ou d'un système afin de détecter les comportements suspects, les anomalies et les tentatives d'intrusion. Son rôle principal est d'identifier les activités malveillantes et de générer des alertes permettant aux administrateurs de prendre les mesures nécessaires [2].

Les IDS peuvent être classés selon plusieurs critères, notamment selon l'emplacement où les données sont analysées et selon la méthode utilisée pour détecter les attaques.

1.3.2.1 Classification selon la source des données

On distingue principalement deux types d'IDS :

- **IDS basé sur l'hôte (HIDS)** : il surveille directement un système ou un serveur en analysant les fichiers système, les journaux d'événements, les processus et les activités des utilisateurs. Il permet ainsi de détecter les comportements suspects affectant une machine spécifique.
- **IDS basé sur le réseau (NIDS)** : il surveille le trafic circulant sur un réseau afin d'identifier des comportements anormaux ou malveillants. Il analyse notamment les caractéristiques des paquets et des flux réseau pour détecter les tentatives d'intrusion.

1.3.2.2 Classification selon la méthode de détection

Selon la technique utilisée pour identifier les attaques, les IDS peuvent également être divisés en plusieurs catégories :

- **IDS basé sur les signatures** : cette approche recherche dans le trafic des motifs correspondant à des attaques connues. Elle est généralement efficace pour détecter des menaces déjà identifiées, mais elle peut être limitée face aux attaques nouvelles ou inconnues.
- **IDS basé sur les anomalies** : il établit un modèle du comportement considéré comme normal, puis détecte les écarts par rapport à ce comportement. Cette approche peut permettre d'identifier des attaques inconnues, mais elle peut également générer davantage de faux positifs.
- **IDS basé sur le Machine Learning** : cette approche utilise des algorithmes d'apprentissage automatique pour analyser les caractéristiques du trafic et classer les activités comme normales ou malveillantes. Elle permet notamment d'exploiter de grandes quantités de données et d'améliorer automatiquement la détection des comportements suspects.

Dans le cadre de ce projet, l'étude porte principalement sur un **NIDS basé sur le Machine Learning**, dont l'objectif est d'analyser les caractéristiques du trafic réseau afin de détecter automatiquement les activités susceptibles de correspondre à des intrusions.

1.4 Solution proposée et objectifs globaux du projet

Afin de répondre à la problématique identifiée, ce projet s'inscrit dans une démarche visant à améliorer la détection des activités potentiellement malveillantes dans les réseaux informatiques.

L'objectif global est de contribuer à la mise en place d'une approche permettant d'identifier automatiquement les comportements suspects et de faciliter ainsi la surveillance de la sécurité du réseau.

Plus précisément, les objectifs du projet sont les suivants :

- Comprendre les principales problématiques liées à la détection des intrusions ;
- Étudier les différentes approches existantes dans le domaine ;
- Analyser les possibilités offertes par l'intelligence artificielle et l'apprentissage automatique ;
- Identifier les méthodes adaptées à la classification des activités réseau ;
- Évaluer la capacité des approches étudiées à distinguer les comportements normaux des comportements malveillants ;
- Contribuer à la conception d'une solution permettant d'améliorer la détection des menaces informatiques.

L'ensemble de ces objectifs vise à proposer une approche cohérente répondant aux enjeux actuels de la cybersécurité et de la protection des réseaux informatiques.

Conclusion

Ce chapitre a permis de présenter le contexte général du projet, de mettre en évidence la problématique liée à la sécurité des réseaux et de donner un aperçu des principales approches existantes dans le domaine de la détection des intrusions. L'étude bibliographique a également permis de mieux comprendre l'intérêt des nouvelles approches dans le domaine de la cybersécurité.

Dans le chapitre suivant, nous nous intéresserons à l'analyse et la spécification des besoins, afin d'identifier les besoins fonctionnels et non fonctionnels du système et de définir les principales exigences auxquelles la solution devra répondre.

Chapitre 2

Analyse et spécification des besoins

Plan

2.1 Identification des acteurs du système 10

2.2 Identification des besoins 10

2.3 Diagramme de GANTT théorique 13

Introduction

Ce chapitre est consacré à l'analyse et à la spécification des besoins liés au système. Il permet de définir les principales fonctionnalités attendues ainsi que les besoins fonctionnels et non fonctionnels auxquels l'application doit répondre.

Dans un premier temps, les différents acteurs et leurs interactions avec le système seront identifiés. Ensuite, l'architecture générale de l'application sera présentée afin de donner une vision globale de son organisation et des principaux composants qui la constituent

2.1 Identification des acteurs du système

L'identification des acteurs permet de déterminer les utilisateurs qui interagissent directement avec le système ainsi que les différentes actions qu'ils peuvent effectuer.

Dans le cadre de ce projet, l'acteur principal identifié est l'utilisateur. Il représente toute personne autorisée à utiliser l'application de détection d'intrusions.

L'utilisateur peut notamment :

- créer un compte à travers la fonctionnalité d'inscription (*Sign Up*) ;
- se connecter à l'application à travers la fonctionnalité de connexion (*Sign In*) ;
- accéder à l'interface principale du système ;
- saisir manuellement les données réseau ;
- importer un fichier CSV contenant les données à analyser ;
- lancer le processus de détection ;
- consulter le résultat de la détection.

Ainsi, l'utilisateur constitue l'acteur principal du système et interagit avec les différentes fonctionnalités de l'application à travers son interface.

2.2 Identification des besoins

L'analyse des besoins permet de déterminer les fonctionnalités que le système doit fournir ainsi que les contraintes auxquelles il doit répondre. Les besoins identifiés sont divisés en deux catégories : les besoins fonctionnels, qui décrivent les services offerts par le système, et les besoins non fonctionnels, qui définissent les qualités et les contraintes du système.

2.2.1 Besoins fonctionnels

Les besoins fonctionnels correspondent aux différentes fonctionnalités que l'application doit offrir à l'utilisateur.

Les principaux besoins fonctionnels sont les suivants :

- **Inscription :** permettre à un nouvel utilisateur de créer un compte.
- **Authentification :** permettre à l'utilisateur enregistré de se connecter à l'application à l'aide de ses identifiants.
- **Accès au système :** permettre à l'utilisateur authentifié d'accéder aux fonctionnalités principales de l'application.
- **Saisie des données :** permettre à l'utilisateur d'entrer manuellement les données nécessaires à l'analyse du trafic réseau.
- **Importation d'un fichier CSV :** permettre à l'utilisateur de charger un fichier contenant les données réseau à analyser.
- **Détection des intrusions :** permettre au système d'analyser les données fournies et d'identifier leur nature.
- **Classification :** permettre au système de classer les données analysées comme étant normales ou susceptibles de correspondre à une attaque.
- **Affichage du résultat :** présenter à l'utilisateur le résultat de
- **Consultation des résultats :** permettre à l'utilisateur de consulter les résultats obtenus après l'analyse.

2.2.2 Besoins non fonctionnels

Les besoins non fonctionnels décrivent les caractéristiques qualitatives que le système doit respecter afin d'assurer une utilisation efficace et fiable de l'application.

Les principaux besoins non fonctionnels sont :

- **Performance :** le système doit fournir les résultats de l'analyse dans un délai raisonnable.
- **Fiabilité :** le système doit fournir des résultats cohérents et limiter les erreurs lors du traitement des données.
- **Sécurité :** les données des utilisateurs et les informations traitées doivent être protégées contre les accès non autorisés.

- **Facilité d'utilisation :** l'interface doit être simple, claire et intuitive afin de faciliter son utilisation.
- **Disponibilité :** l'application doit être accessible et fonctionnelle lorsque l'utilisateur souhaite effectuer une analyse.
- **Maintenabilité :** le système doit être organisé de manière à faciliter sa maintenance, sa correction et son évolution.
- **Évolutivité :** l'application doit pouvoir être améliorée ultérieurement par l'ajout de nouvelles fonctionnalités ou de nouvelles méthodes de détection.
- **Compatibilité :** l'application doit pouvoir fonctionner correctement dans un environnement informatique adapté et avec les navigateurs couramment utilisés.

2.2.3 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation présente les principales interactions entre l'utilisateur et le système de détection d'intrusions.

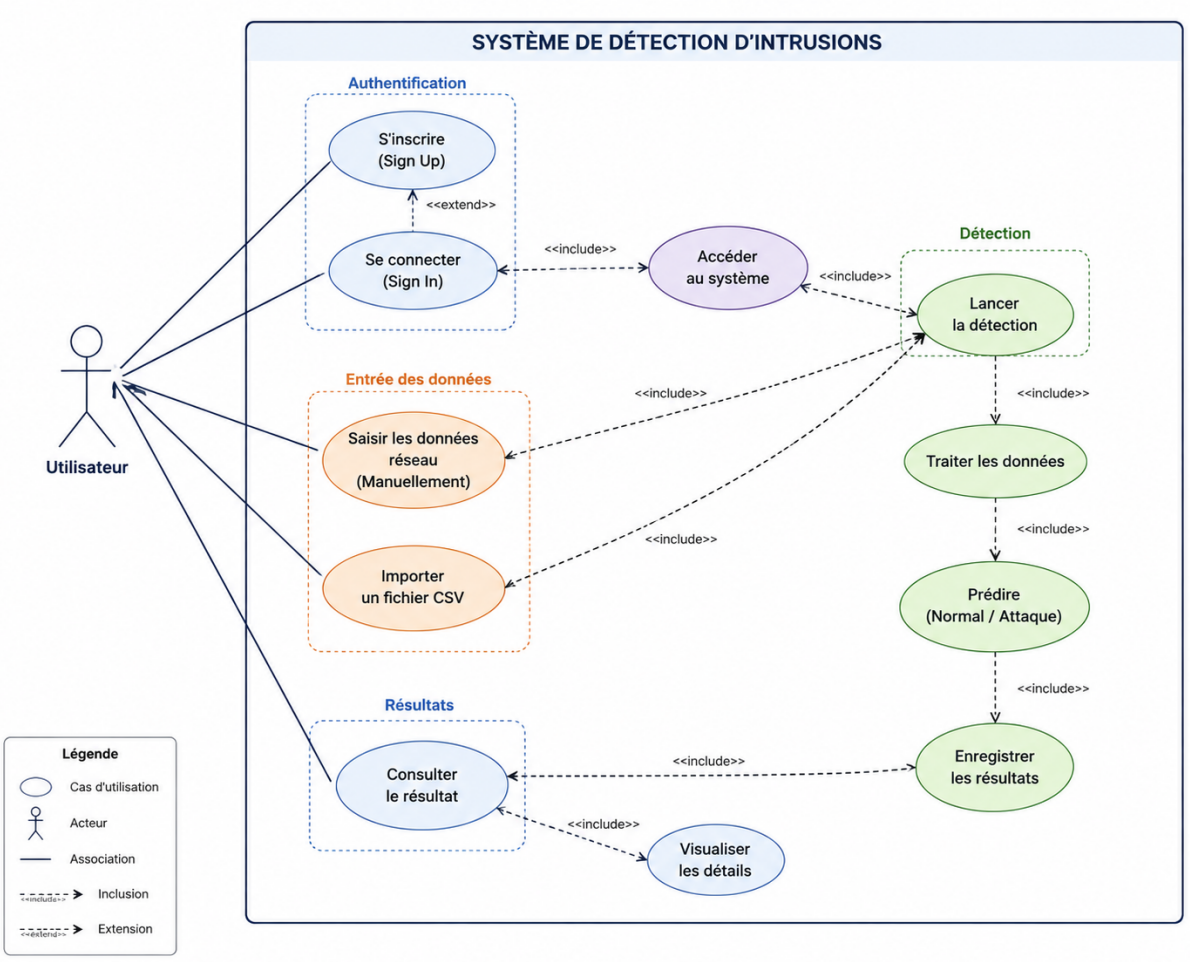


Figure 2.1 : Diagramme de cas d'utilisation du système

Ce diagramme montre que l'utilisateur peut s'inscrire, se connecter, accéder au système, saisir ou importer les données réseau, lancer la détection et consulter les résultats.

2.2.4 Diagramme de séquence

Le diagramme de séquence décrit l'ordre des interactions entre l'utilisateur et les différents composants du système lors de la détection d'une intrusion.

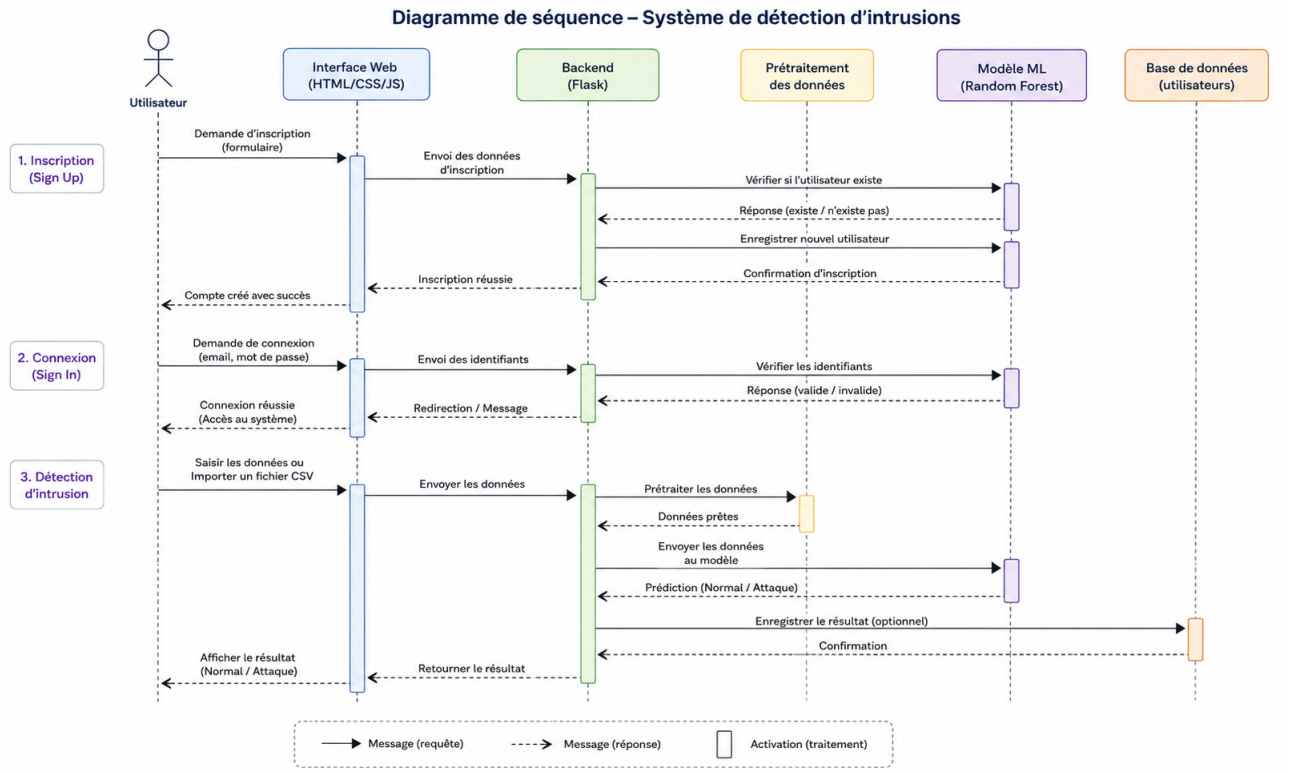


Figure 2.2 : Diagramme de séquence de la détection d'intrusion

Ce diagramme illustre le déroulement de la détection, depuis l'envoi des données par l'utilisateur jusqu'à l'affichage du résultat.

2.3 Diagramme de GANTT théorique

Le diagramme de Gantt ci-dessous présente les différentes étapes de réalisation du projet, depuis l'étude du sujet jusqu'à la validation finale et la préparation de la soutenance.

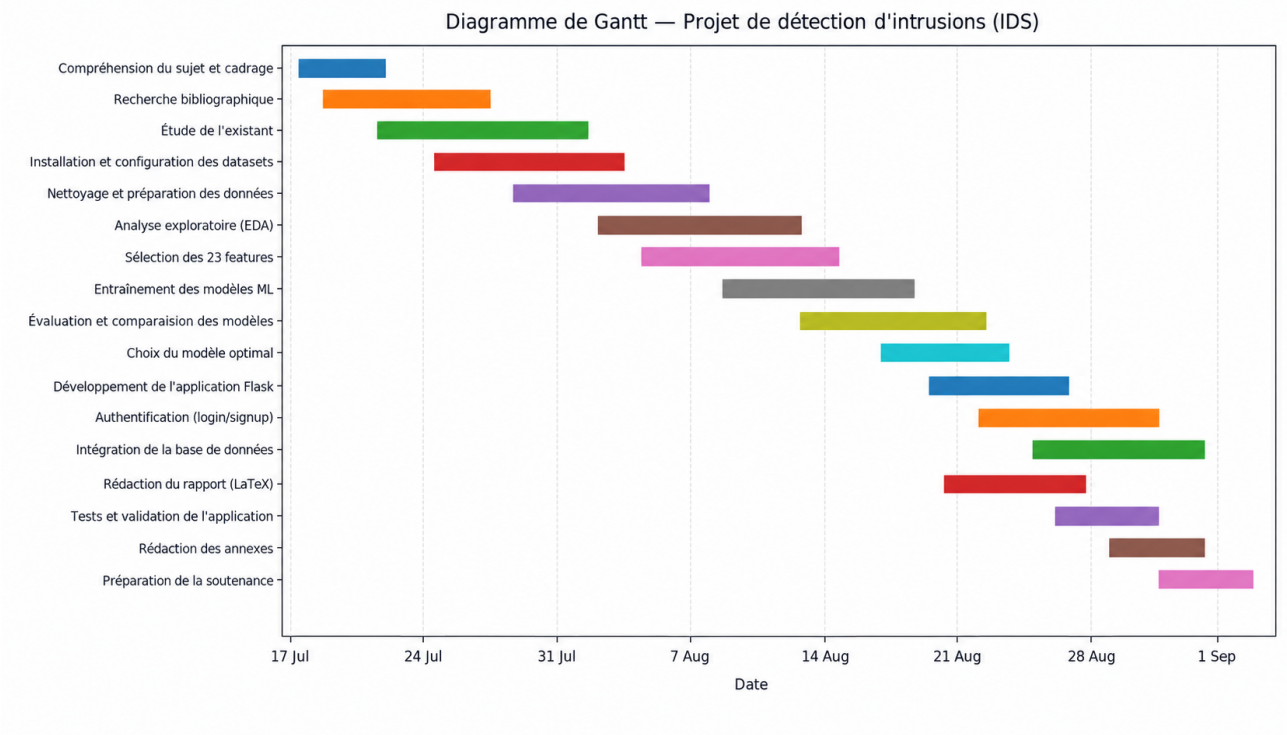


Figure 2.3 : Diagramme de Gantt du projet

Conclusion

Ce chapitre a permis d'identifier les différents acteurs du système ainsi que les besoins fonctionnels et non fonctionnels auxquels l'application doit répondre. L'analyse réalisée a permis de définir les principales fonctionnalités attendues et de mieux cerner les exigences du système.

Sur la base de ces besoins, le chapitre suivant sera consacré à la conception architecturale et au choix des technologies, afin de définir l'organisation générale de la solution et de présenter les technologies retenues pour sa réalisation.

Chapitre 3

Conception et choix technologiques

Plan

3.1 Conception architecturale du système 16

3.2 Préparation et prétraitement des données 20

3.3 Principes du Machine Learning pour la Détection d'Intrusions 23

3.4 Techniques d'Évaluation des Modèles 24

3.5 Choix du Modèle Optimal 26

Introduction

Ce chapitre présente la conception générale de la solution ainsi que les principaux éléments nécessaires à sa réalisation. Il aborde l'architecture du système, le dataset utilisé, sa préparation, les modèles de Machine Learning étudiés ainsi que les technologies retenues pour le développement de l'application.

3.1 Conception architecturale du système

3.1.1 Architecture générale

L'architecture proposée repose sur plusieurs composants qui assurent la communication entre l'utilisateur, l'application web et le modèle de Machine Learning. Le système permet de traiter les données réseau provenant soit d'une saisie manuelle, soit d'un fichier CSV, puis de les soumettre au modèle afin d'obtenir une classification.

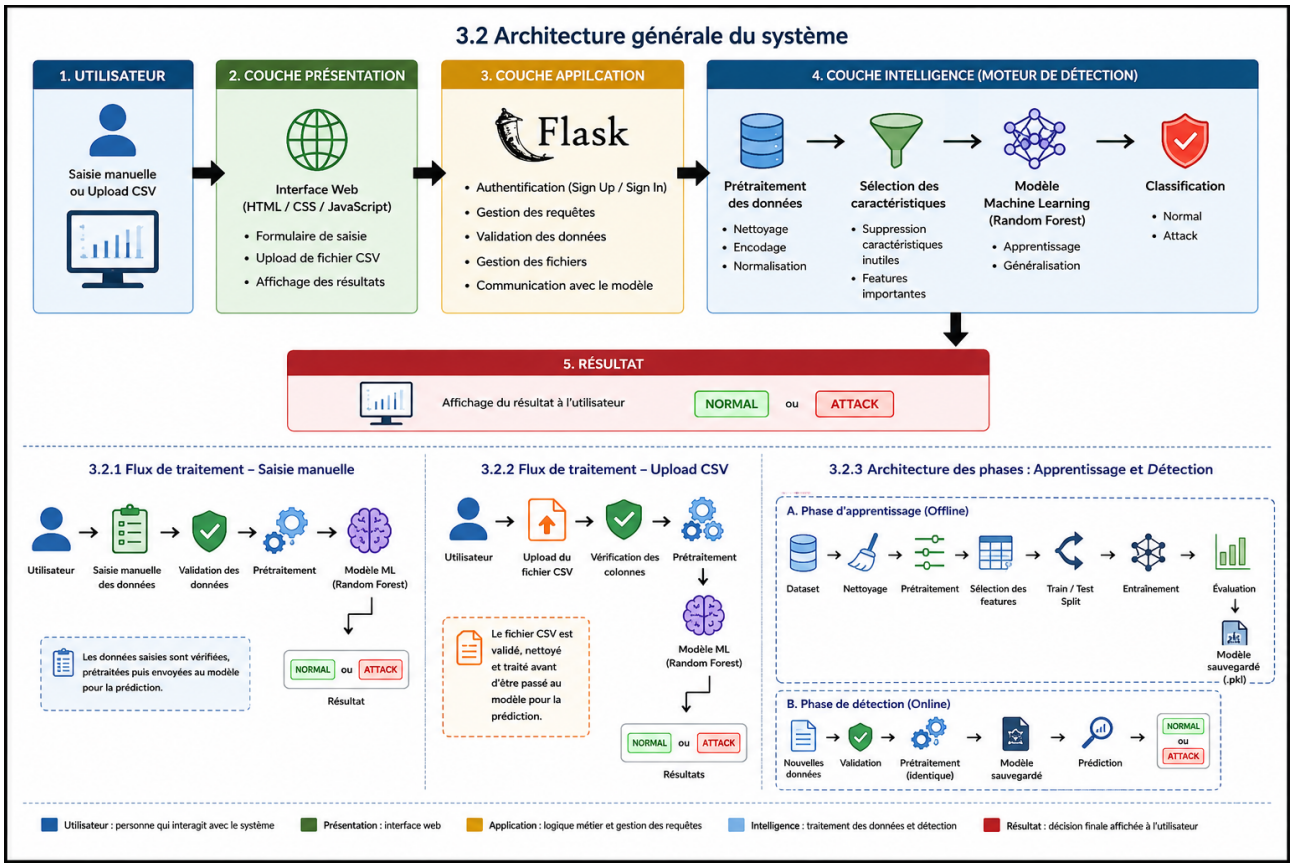


FIGURE 3.1 : Architecture générale du système de détection d'intrusions

3.1.1.1 Couche utilisateur

L'utilisateur constitue le point d'entrée du système. Il peut interagir avec l'application à travers une saisie manuelle des données réseau ou par l'importation d'un fichier CSV contenant les données à analyser. Après le traitement, le résultat de la détection lui est présenté à travers l'interface web.

3.1.1.2 Couche présentation

La couche de présentation correspond à l'interface web de l'application. Elle permet à l'utilisateur de s'inscrire, de se connecter, de saisir les données nécessaires à la détection ou d'importer un fichier CSV. Elle assure également l'affichage des résultats de l'analyse.

3.1.1.3 Couche application

La couche application est assurée par le framework Flask. Elle joue le rôle d'intermédiaire entre l'interface utilisateur et le modèle de Machine Learning. Elle prend en charge l'authentification, la réception des requêtes, la validation des données, la gestion des fichiers ainsi que la communication avec le modèle de détection.

3.1.1.4 Couche intelligence et détection

Cette couche constitue le cœur du système de détection. Elle prend en charge la préparation des données avant leur transmission au modèle. Les données sont nettoyées et transformées afin de respecter le format attendu par le modèle. Les caractéristiques nécessaires à la prédiction sont ensuite utilisées par le modèle de Machine Learning.

Dans le cadre de ce projet, le modèle Random Forest est utilisé comme moteur de détection. Il analyse les caractéristiques du trafic réseau et produit une prédiction permettant de distinguer le trafic normal du trafic considéré comme une attaque.

3.1.1.5 Résultat de la détection

Après l'analyse des données, le modèle retourne une prédiction. Le résultat est ensuite transmis à l'application Flask puis affiché à l'utilisateur à travers l'interface web. Le trafic est ainsi classé selon les catégories définies dans le système, notamment *Normal* ou *Attack*.

3.1.1.6 Flux de traitement des données

Les données sont d'abord saisies ou importées sous forme de fichier CSV. Elles sont ensuite vérifiées et prétraitées avant d'être transmises au modèle Random Forest. Enfin, le modèle effectue la prédiction et affiche le résultat sous forme de trafic normal ou d'attaque.

3.1.2 Présentation du dataset

3.1.2.1 Description du dataset

Dans le cadre de ce projet, le jeu de données utilisé est *Thursday-WorkingHours-Morning-WebAttacks.pcap_ISCX.csv*, issu du jeu de données CICIDS2017 [3]. Il contient des informations

représentant différents flux de trafic réseau et permet d'identifier des activités normales ainsi que plusieurs types d'attaques.

Le dataset contient notamment des caractéristiques liées à la durée des flux, aux paquets échangés, à leurs longueurs ainsi qu'aux intervalles entre les paquets. Ces informations sont utilisées comme entrées pour le modèle de détection.

3.1.2.2 Structure et caractéristiques des données

Le jeu de données est constitué de plusieurs caractéristiques (*features*) décrivant le comportement du trafic réseau. Chaque ligne représente un flux réseau et chaque colonne correspond à une caractéristique mesurée.

Le dataset initial contient 79 colonnes, dont la variable cible *Label*. Après séparation de cette variable, 78 caractéristiques sont disponibles pour l'analyse et la classification.

Quelques exemples de caractéristiques utilisées sont présentés dans le tableau suivant.

Tableau 3.1 : Exemples de caractéristiques du dataset

Caractéristique	**Description**
Flow Duration	Durée du flux réseau
Total Length of Fwd Packets	Longueur totale des paquets transmis
Fwd Packet Length Max	Taille maximale des paquets transmis
Flow Bytes/s	Nombre d'octets transmis par seconde
Flow Packets/s	Nombre de paquets transmis par seconde
Packet Length Mean	Longueur moyenne des paquets
Packet Length Std	Écart-type de la longueur des paquets
Average Packet Size	Taille moyenne des paquets
Init_Win_bytes_forward	Taille initiale de la fenêtre TCP
Init_Win_bytes_backward	Taille initiale de la fenêtre TCP inverse

3.1.2.3 Répartition des classes

Après les étapes de nettoyage, le dataset contient 164179 observations. La variable cible *Label* comprend une classe normale (*BENIGN*) et plusieurs catégories d'attaques.

Tableau 3.2 : Répartition des classes dans le dataset

Classe	**Nombre d'observations**
BENIGN	162036
Web Attack – Brute Force	1470
Web Attack – XSS	652
Web Attack – Sql Injection	21
Total	**164179**

La répartition montre un déséquilibre important entre le trafic normal et le trafic malveillant. Pour simplifier la tâche de classification, les différentes catégories d'attaques ont été regroupées dans une seule classe "Attack", tandis que la classe "BENIGN" représente le trafic normal.

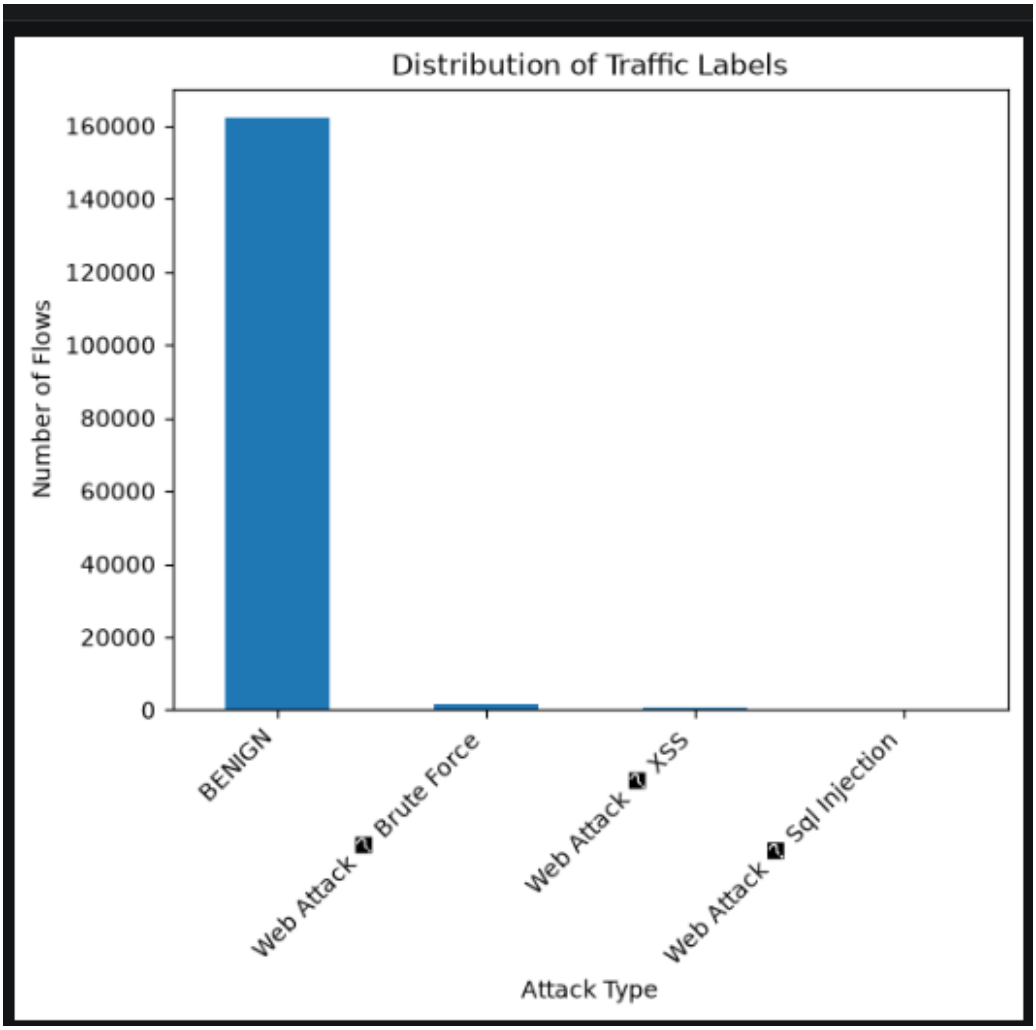


Figure 3.2 : Répartition des classes dans le dataset

La figure 3.2 met en évidence un déséquilibre important entre les différentes classes, avec une forte majorité de trafic normal par rapport au trafic malveillant.

3.1.3 Problèmes dans les Données

Lors de l'analyse du jeu de données, plusieurs problèmes ont été identifiés :

- **Déséquilibre des classes :** le nombre de connexions normales était beaucoup plus élevé que celui des connexions malveillantes. Pour limiter l'impact de ce déséquilibre, nous avons utilisé une séparation stratifiée des données avec *stratify=y* et avons appliqué *class_ weight="balanced"* au modèle Random Forest afin d'accorder davantage d'importance à la classe minoritaire.
- **Valeurs manquantes et infinies :** les valeurs infinies ont été remplacées par des valeurs manquantes, puis les observations contenant des valeurs manquantes ont été supprimées.
- **Doublons :** les observations en double ont été supprimées afin d'obtenir un jeu de données plus fiable.
- **Noms des colonnes :** les espaces inutiles présents dans les noms des colonnes ont été supprimés afin de faciliter la manipulation des caractéristiques.
- **Nombre de caractéristiques :** parmi les caractéristiques disponibles, nous avons sélectionné **23 caractéristiques** adaptées à notre modèle pour l'entraînement final.

3.2 Préparation et prétraitement des données

3.2.1 Nettoyage des données

Avant l'entraînement des modèles, les données ont été nettoyées afin d'améliorer leur qualité et d'éviter les erreurs lors de l'apprentissage. Cette étape consiste notamment à traiter les valeurs manquantes ainsi que les valeurs infinies présentes dans certaines caractéristiques.

Après le nettoyage, les données obtenues sont utilisées pour les étapes suivantes de préparation et d'apprentissage.

3.2.1.1 Imputation des Valeurs Manquantes

Dans un jeu de données utilisé pour la détection d'intrusions, certaines valeurs peuvent être absentes, ce qui peut affecter la qualité des données et les performances des algorithmes. Plusieurs méthodes peuvent être utilisées pour gérer ces valeurs manquantes :

— **Suppression des lignes/colonnes :** Cette méthode consiste à supprimer les observations ou les colonnes contenant des valeurs manquantes. Elle est particulièrement adaptée lorsque le nombre de valeurs manquantes est faible par rapport à la taille globale du jeu de données.

— **Imputation des valeurs manquantes :** Cette méthode consiste à remplacer les valeurs manquantes par une valeur estimée, telle que la moyenne, la médiane ou le mode. Elle peut être utilisée lorsque la suppression des observations entraînerait une perte importante de données.

— **Utilisation de modèles prédictifs :** Des techniques plus avancées peuvent également être utilisées afin d'estimer les valeurs manquantes à partir des autres caractéristiques du jeu de données.

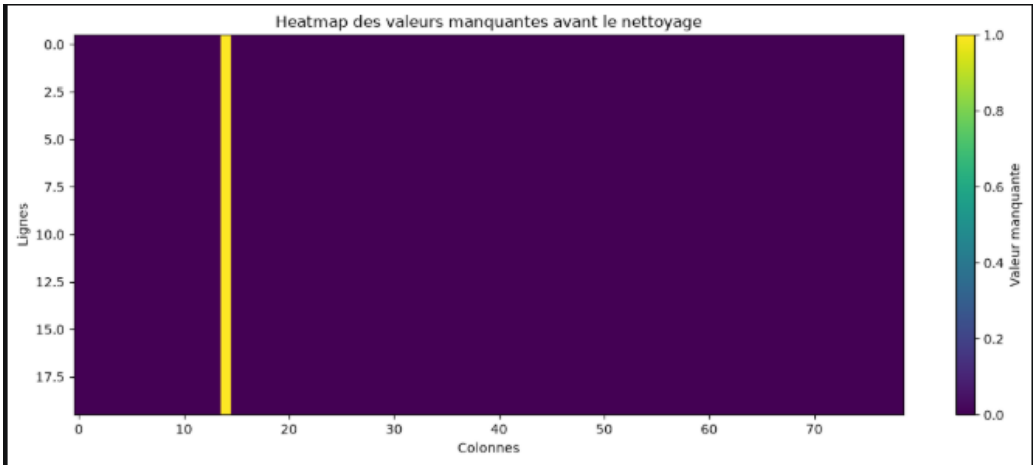


Figure 3.3 : Heatmap des valeurs manquantes dans le dataset

Dans notre projet de détection d'intrusions, l'analyse du jeu de données a montré la présence de valeurs manquantes uniquement dans la colonne `Flow Bytes/s`, avec un total de 20 valeurs manquantes. Étant donné que ce nombre est très faible par rapport à la taille globale du jeu de données, nous avons choisi d'utiliser la méthode de suppression des lignes contenant ces valeurs.

Cette opération a été réalisée à l'aide de la fonction `dropna()` de la bibliothèque Pandas. Pour visualiser la répartition des valeurs manquantes avant leur suppression, nous avons utilisé une heatmap réalisée avec la bibliothèque Matplotlib. Après la suppression des valeurs manquantes, nous avons vérifié à nouveau le jeu de données afin de nous assurer que les données sont complètes. La figure suivante présente l'état du jeu de données après l'étape de nettoyage.

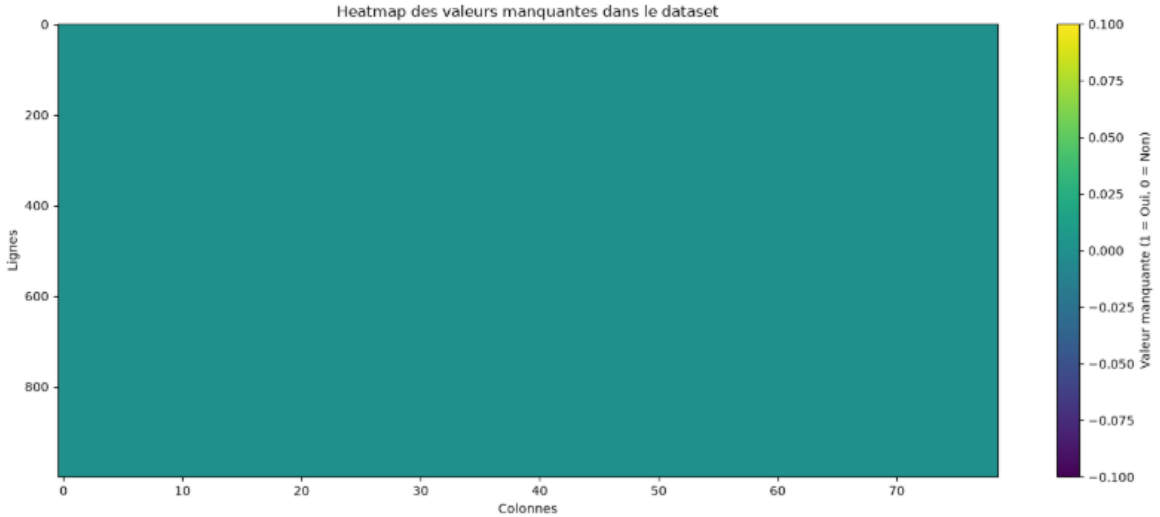


Figure 3.4 : État des valeurs manquantes après le nettoyage

— **Traitement des valeurs infinies :** Lors de l'analyse du jeu de données, certaines variables peuvent contenir des valeurs infinies positives ou négatives ($+\infty$ et $-\infty$), notamment à la suite de certaines opérations de calcul effectuées sur les caractéristiques du trafic réseau. Ces valeurs peuvent perturber les étapes d'apprentissage et entraîner des erreurs lors de l'utilisation des algorithmes de Machine Learning.

Dans notre projet, nous avons donc traité ces valeurs avant l'entraînement du modèle. À l'aide de la fonction `replace()` de la bibliothèque Pandas, les valeurs $+\infty$ et $-\infty$ ont d'abord été remplacées par des valeurs manquantes (`NaN`). Ensuite, la fonction `dropna()` a permis de supprimer les observations contenant ces valeurs.

Cette étape permet ainsi d'obtenir un jeu de données numérique plus propre et compatible avec l'algorithme de classification utilisé pour la détection des intrusions.

3.2.1.2 Sélection des caractéristiques (Feature Selection)

La sélection des caractéristiques est une étape essentielle en Machine Learning qui permet de réduire le nombre de variables utilisées pour l'entraînement du modèle. Elle permet ainsi de réduire la complexité des données, d'accélérer les calculs et de faciliter l'interprétation des résultats. Dans le domaine de la détection d'intrusions, cette étape permet de conserver les caractéristiques les plus pertinentes pour décrire le comportement du trafic réseau.

Dans notre projet, le jeu de données contient un grand nombre de caractéristiques décrivant les différents flux réseau. Afin de construire un modèle plus simple et adapté à notre problème, nous avons sélectionné un ensemble de 23 caractéristiques numériques liées aux propriétés du trafic réseau, telles que la durée du flux, le nombre et la taille des paquets, les intervalles entre les paquets ainsi que certaines informations relatives aux en-têtes.

La sélection des caractéristiques a été réalisée directement à partir de la liste des variables retenues dans notre code Python. La matrice des données d'entrée a ensuite été construite à l'aide de la commande `X = df[features]`, ce qui permet de limiter l'apprentissage du modèle aux 23 caractéristiques sélectionnées.

Afin de compléter cette sélection, nous avons également utilisé l'importance des caractéristiques fournie par l'algorithme Random Forest. Cette analyse permet de classer les variables selon leur contribution aux décisions du modèle. Les résultats obtenus montrent que `Init_Win_bytes_backward` est la caractéristique la plus importante avec une valeur de 0,1868, suivie de `Flow IAT Std` avec 0,0877 et de `Fwd Header Length.1` avec 0,0839.

Cette analyse nous permet ainsi d'identifier les caractéristiques les plus influentes dans la détection du trafic malveillant et de mieux comprendre le fonctionnement du modèle utilisé.

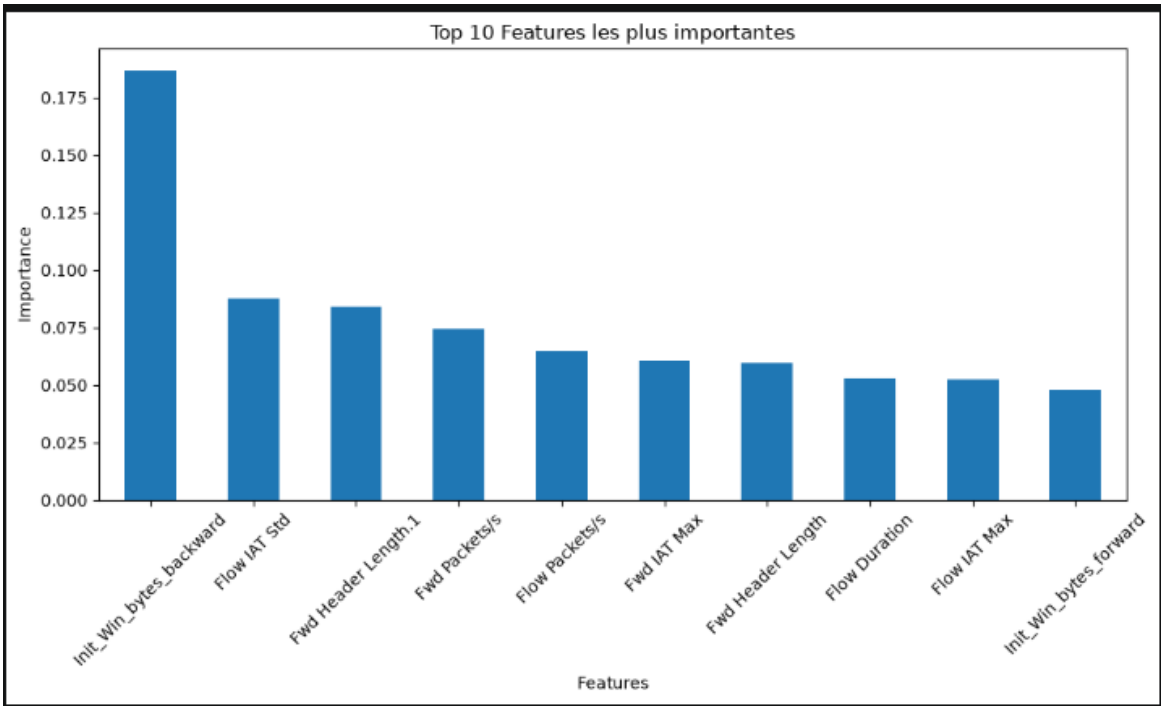


Figure 3.5 : Top 10 Features les plus importantes

3.3 Principes du Machine Learning pour la Détection d'Intrusions

Le Machine Learning repose sur la capacité des algorithmes à apprendre à partir de données sans être explicitement programmés pour chaque scénario. Dans le contexte de la cybersécurité, il permet d'analyser les données réseau et de détecter automatiquement les comportements suspects. Les techniques de Machine Learning appliquées aux IDS peuvent être regroupées en plusieurs catégories :

— **Apprentissage supervisé** : cette approche repose sur l'utilisation de données étiquetées contenant des exemples d'attaques et de trafic normal. Le modèle apprend à reconnaître les différentes classes afin de classifier de nouvelles observations. Dans notre projet, nous avons utilisé cette approche avec les algorithmes **Decision Tree** et **Random Forest**.

— **Apprentissage non supervisé** : cette approche ne nécessite pas de données étiquetées. Des algorithmes comme K-means ou DBSCAN permettent d'identifier des comportements inhabituels.

— **Apprentissage semi-supervisé** : cette méthode combine un petit ensemble de données étiquetées avec un grand volume de données non étiquetées.

— **Apprentissage par renforcement** : cette approche permet au système d'améliorer ses décisions à partir de récompenses et de pénalités.

3.3.1 Les Algorithmes de Classification Utilisés

Afin de réaliser la détection des intrusions dans notre projet, nous avons utilisé des algorithmes d'apprentissage supervisé. Ces algorithmes permettent d'apprendre à partir de données étiquetées afin de distinguer le trafic normal du trafic malveillant. Nous avons choisi deux algorithmes de classification : le *Decision Tree* et le *Random Forest*.

3.3.1.1 Decision Tree (DT)

Le *Decision Tree* est un algorithme d'apprentissage supervisé basé sur une succession de règles de décision [4]. Il permet de classer les connexions réseau en fonction de leurs différentes caractéristiques. Dans notre projet, il est utilisé pour distinguer les connexions normales des connexions malveillantes.

3.3.1.2 Random Forest (RF)

Le *Random Forest* est un algorithme d'apprentissage supervisé constitué d'un ensemble de plusieurs arbres de décision. Il combine les résultats de ces arbres afin d'obtenir une classification plus robuste. Dans notre projet, il est utilisé pour améliorer la détection des connexions malveillantes [5].

Tableau 3.3 : Comparaison des algorithmes de classification utilisés

Algorithme	**Avantages**	**Inconvénients**
Decision Tree	Simple et facile à interpréter	Risque de surapprentissage
Random Forest	Robuste et performant	Plus complexe et plus coûteux en calcul

3.4 Techniques d'Évaluation des Modèles

3.4.1 Accuracy, Precision, Recall et F1-Score

Dans cette section, nous détaillons les différentes métriques utilisées pour évaluer les performances du modèle de classification. Ces métriques sont essentielles pour mesurer la capacité du modèle à distinguer correctement le trafic normal du trafic malveillant, tout en prenant en compte les différentes erreurs de classification.

1. Accuracy (Précision globale) : L'Accuracy est une métrique classique qui mesure la proportion des prédictions correctement classées parmi l'ensemble des prédictions réalisées. Elle est calculée comme suit :

$$Accuracy = \frac{\text{Nombre de prédictions correctes}}{\text{Nombre total d'échantillons}}$$

Dans notre étude, l'Accuracy permet d'avoir une première indication sur les performances globales du modèle de détection d'intrusions.

Bien que l'Accuracy soit une métrique utile, elle peut être trompeuse lorsque les classes sont déséquilibrées. Dans notre cas, le jeu de données contient une différence importante entre le trafic normal et le trafic malveillant. C'est pourquoi il est nécessaire de compléter cette mesure par d'autres critères tels que la Precision, le Recall et le F1-Score.

2. Precision (Précision) : La Precision mesure la proportion des vrais positifs parmi les instances que le modèle a classées comme positives. Autrement dit, parmi toutes les prédictions indiquant la présence d'une attaque, elle permet de déterminer combien étaient réellement des attaques. Elle est définie par la formule suivante :

$$Precision = \frac{VP}{VP + FP}$$

où VP représente les vrais positifs et FP les faux positifs.

Dans notre projet, une Precision élevée signifie que le modèle génère peu de fausses alertes lorsqu'il identifie un trafic comme étant malveillant.

3. Recall (Rappel) : Le Recall mesure la proportion des vrais positifs parmi les instances réellement positives. Autrement dit, parmi toutes les attaques présentes dans les données, il indique la proportion que le modèle a correctement détectée. Il est calculé comme suit :

$$Recall = \frac{VP}{VP + FN}$$

où VP représente les vrais positifs et FN les faux négatifs.

Dans le contexte de la détection d'intrusions, le Recall est particulièrement important, car un faux négatif correspond à une attaque réelle qui n'a pas été détectée par le modèle. Un Recall élevé indique donc que le modèle est capable d'identifier une grande partie des activités malveillantes.

4. F1-Score : Le F1-Score représente la moyenne harmonique entre la Precision et le Recall. Il permet d'obtenir un équilibre entre ces deux métriques et constitue une mesure particulièrement utile lorsque les classes sont déséquilibrées. Il est défini par la formule suivante :

$$F1\text{-}Score = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

Dans notre projet, le F1-Score permet ainsi d'évaluer la capacité globale du modèle Random Forest à détecter correctement les attaques tout en limitant les fausses alertes.

Modèle	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)
Random Forest	99.9360	96.5753	98.6014	97.5779
Decision Tree	99.9178	95.6818	98.1352	96.8930

Tableau 3.4 : Comparaison des performances des modèles

3.5 Choix du Modèle Optimal

Après comparaison des modèles, le **Random Forest** a obtenu les meilleures performances avec une Accuracy de **99,9360%** et un F1-Score de **97,5779%**. Il présente également un Recall élevé, ce qui est important pour détecter efficacement les attaques.

Ainsi, le **Random Forest** a été retenu comme modèle final pour notre système de détection d'intrusions.

Conclusion

Dans ce chapitre, nous avons préparé les données, entraîné et comparé plusieurs modèles de classification, puis retenu le **Random Forest** comme modèle optimal. Le chapitre suivant sera consacré à la **Choix techniques et à la réalisation de l'application de détection d'intrusions**.

Chapitre 4

Réalisation

Plan

4.1 Choix techniques 28

4.2 l'environnement de développement 30

4.3 Travail réalisé 31

Introduction

Après avoir sélectionné le modèle optimal, ce chapitre présente la **réalisation** de notre application de détection d'intrusions. Nous décrirons son architecture, ses principales fonctionnalités ainsi que son interface utilisateur.

4.1 Choix techniques

La réalisation de cette application a nécessité l'utilisation de plusieurs outils et technologies adaptés aux différentes étapes du projet.

4.1.1 Flask

Flask est un micro-framework Python utilisé pour développer des applications web [6]. Dans notre projet, il permet de créer le serveur web, de gérer les différentes routes et de relier l'interface utilisateur au modèle de Machine Learning.



Figure 4.1 : Flask

4.1.2 sqlite

SQLite est un système de gestion de base de données relationnelle léger et autonome. Il permet de stocker et de gérer les données directement dans un fichier, sans nécessiter de serveur externe. **SQLite** est utilisée dans notre projet pour assurer la gestion de l'authentification des utilisateurs, notamment lors de la connexion et de la déconnexion.



Figure 4.2 : sqlite

4.1.3 HTML

HTML est utilisé dans notre projet pour structurer les différentes pages de l'application, notamment les interfaces de connexion, d'accueil et de détection d'intrusions.



Figure 4.3 : HTML

4.1.4 CSS

CSS est utilisé dans notre projet pour personnaliser l'apparence de l'application, notamment les couleurs, les styles, les espacements et la disposition des différents éléments de l'interface.



Figure 4.4 : CSS

4.1.5 Bootstrap

Bootstrap est utilisé dans notre projet pour concevoir une interface moderne, claire et responsive. Il facilite la mise en forme des pages et l'adaptation de l'application aux différents écrans.



Figure 4.5 : Bootstrap

4.1.6 Javascript

JavaScript : langage de programmation permettant de rendre les pages web interactives et dynamiques. Dans notre projet, il est utilisé notamment avec **jQuery** pour la manipulation du DOM et la gestion des interactions dans l'interface.



Figure 4.6 : Javascript

4.2 l'environnement de développement

Visual Studio Code est un environnement de développement léger et polyvalent, utilisé pour écrire, modifier et organiser le code de notre application. Dans le cadre de notre projet de détection d'intrusions, il a été utilisé principalement pour développer l'application web avec Flask, intégrer les différentes pages HTML, CSS et JavaScript, ainsi que pour organiser les fichiers et tester le fonctionnement global de l'application. Son interface et ses nombreuses extensions facilitent également le développement et le débogage du projet.



Figure 4.7 : Vs code

Jupyter Notebook est un environnement interactif permettant d'exécuter du code, d'analyser les données et de visualiser les résultats étape par étape. Dans notre projet, il a été utilisé principalement lors de la phase d'analyse et de prétraitement du jeu de données, notamment pour le nettoyage des données, l'identification des valeurs manquantes, l'exploration des caractéristiques et la préparation des données pour l'entraînement des modèles de Machine Learning. Il a ainsi facilité l'expérimentation et l'analyse des résultats avant leur intégration dans l'application.



Figure 4.8 : Jupyter Notebook

LaTeX est un système de composition de documents utilisé pour produire des documents

scientifiques et techniques avec une mise en page structurée et professionnelle. Dans notre projet, il a été utilisé pour rédiger et organiser la documentation, notamment les différents chapitres, sections, tableaux, figures et références bibliographiques. Il permet ainsi d'assurer une présentation claire et cohérente du rapport.

FIGURE 4.9 : latex

4.3 Travail réalisé

4.3.1 Page d'accueil

La page d'accueil de notre application constitue le premier point de contact avec l'utilisateur. Elle présente une interface moderne et épurée, dotée d'un arrière-plan dynamique illustrant une intelligence artificielle sous forme d'un visage robotique connecté, symbolisant l'analyse intelligente et automatisée du trafic réseau assurée par notre système. Le message d'accueil *"Welcome To Dashboard"* accompagné du sous-titre *"Machine Learning-Based Intrusion Detection System"* met immédiatement en avant la vocation principale de l'application : la détection d'intrusions basée sur l'apprentissage automatique. La barre de navigation, simple et intuitive, offre un accès direct à la page d'inscription (*Signup*) ainsi qu'un bouton permettant de basculer entre le mode clair et le mode sombre, offrant ainsi une expérience utilisateur confortable et personnalisable.

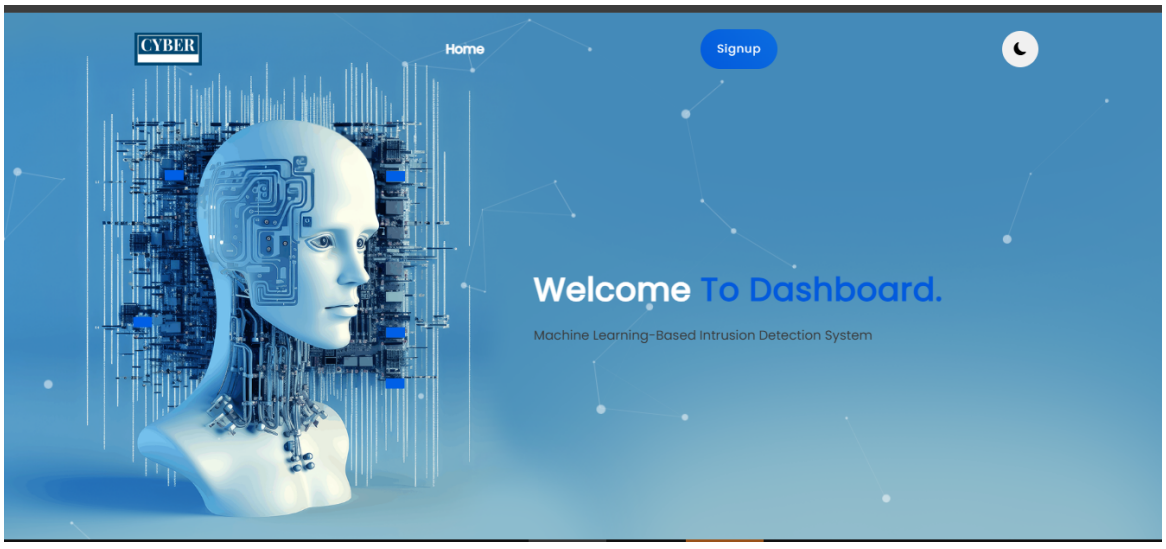


FIGURE 4.10 : Page d'accueil de l'application

4.3.2 Section "About"

Juste au-dessous de la page d'accueil, une section *"About"* présente sous forme de trois cartes les principaux piliers du projet : le dataset **CICIDS2017** utilisé pour l'entraînement du modèle, les **Key**

Features de l'application (classification en temps réel via Random Forest, authentification sécurisée, prédiction instantanée), et le framework **Flask** utilisé pour le développement web.

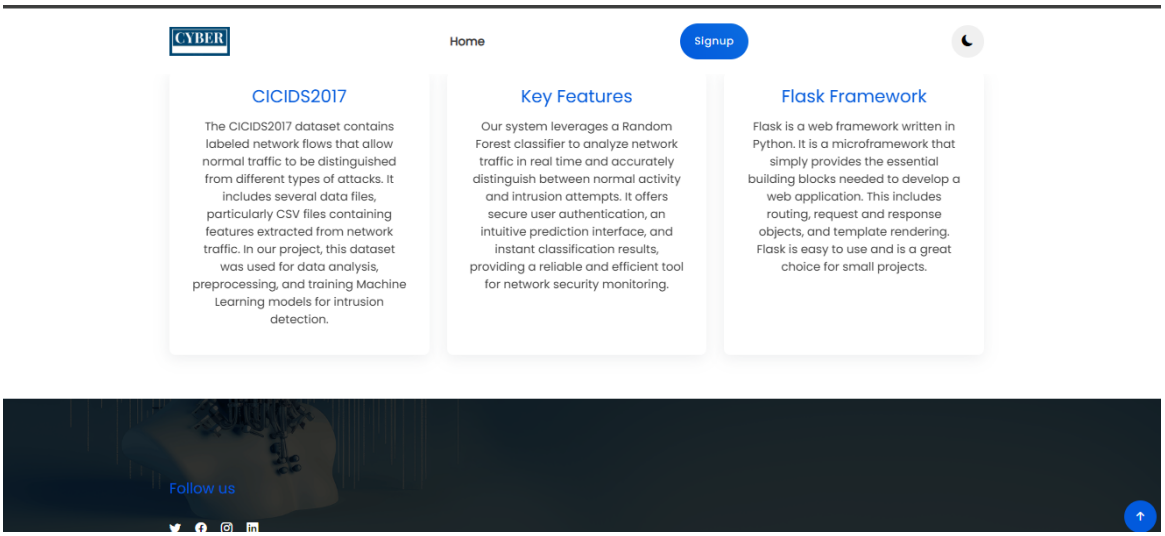


FIGURE 4.11 : Section "About" de l'application

4.3.3 Pages d'authentification

Les pages *Sign Up* et *Sign In* permettent respectivement l'inscription et la connexion des utilisateurs à l'application. La page d'inscription requiert un nom d'utilisateur, un nom complet, une adresse e-mail, un numéro de téléphone et un mot de passe, tandis que la page de connexion se limite au nom d'utilisateur et au mot de passe, avec une option *"Remember Me"* et un lien de récupération du mot de passe.

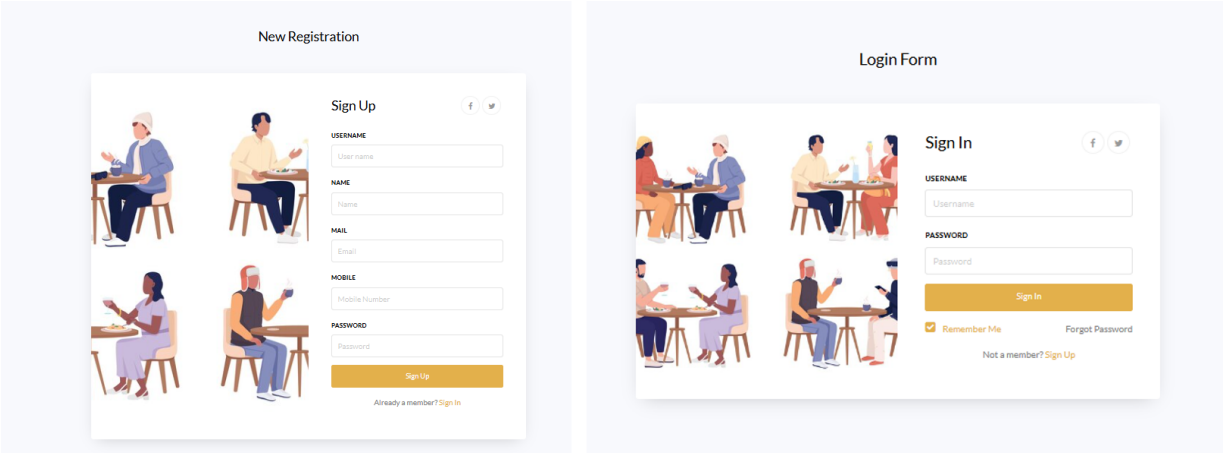


FIGURE 4.12 : Pages d'inscription et de connexion

4.3.4 Interface de prédiction

Une fois connecté, l'utilisateur accède au tableau de bord principal offrant deux modes de détection : la *Prédiction Manuelle*, permettant de saisir directement les caractéristiques d'un flux réseau, et l'*Import CSV*, permettant d'analyser un fichier de capture réseau complet. La page de prédiction manuelle présente un formulaire détaillé regroupant l'ensemble des caractéristiques du trafic (durée

du flux, longueur des paquets, débit, etc.), extraites du dataset CICIDS2017, que le modèle Random Forest utilise pour classifier le trafic comme normal ou potentiellement malveillant.

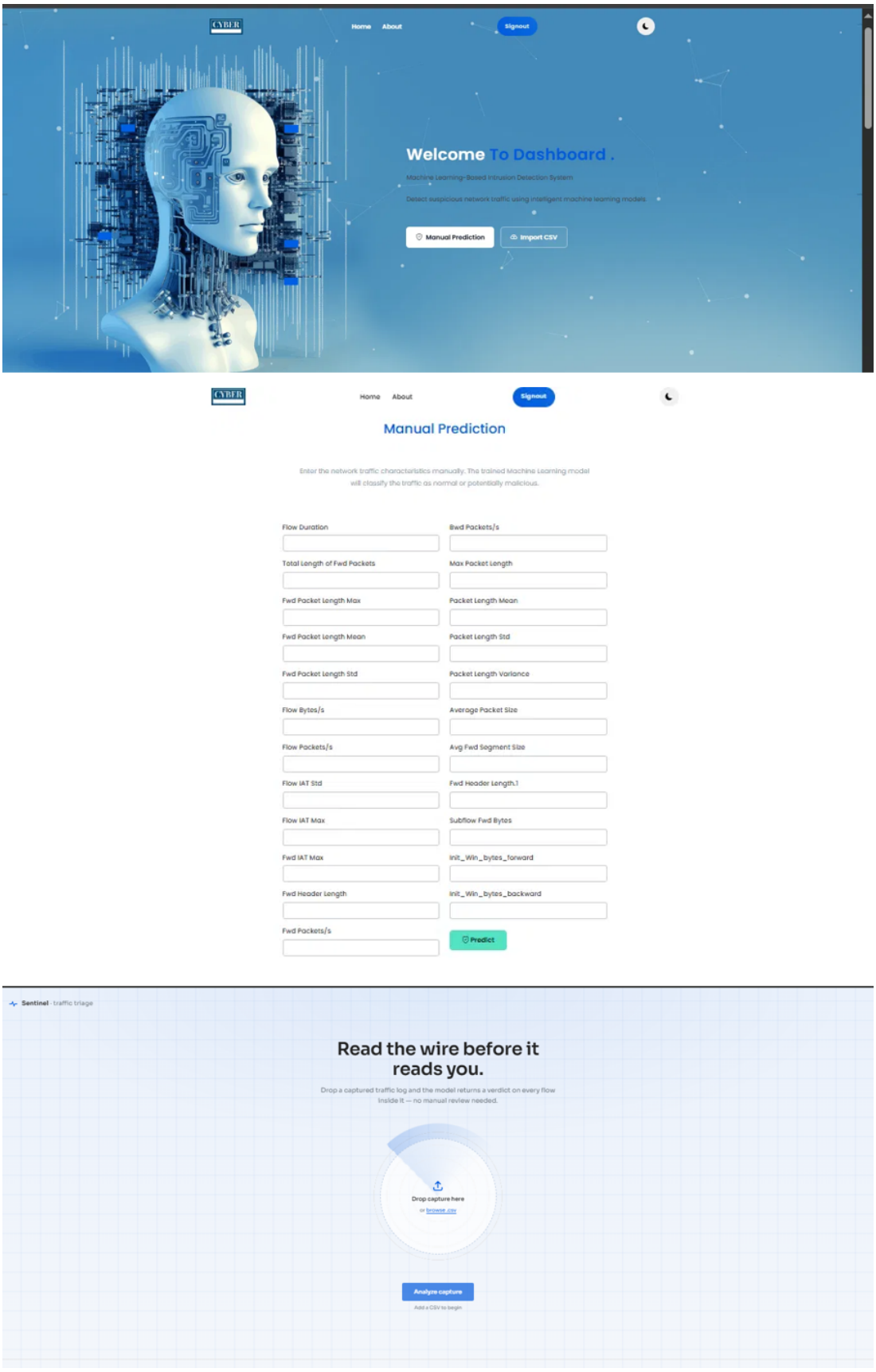


Figure 4.13 : Interfaces de prédiction de l'application

4.3.5 Résultat de la prédiction

Après l'envoi du fichier capturé, le résultat de l'analyse s'affiche instantanément dans une nouvelle section intitulée *"Analysis result"*. Cette dernière présente un badge de verdict global (*"Threats Detected"* en cas d'attaque détectée), ainsi qu'un résumé statistique du nombre total de flux analysés, du nombre de flux classés comme normaux et du nombre de flux identifiés comme attaques. Un détail par flux est également affiché, indiquant pour chacun le résultat de la classification (*Normal* ou *Attack*), permettant à l'utilisateur d'avoir une vision claire et précise des résultats obtenus par le modèle.

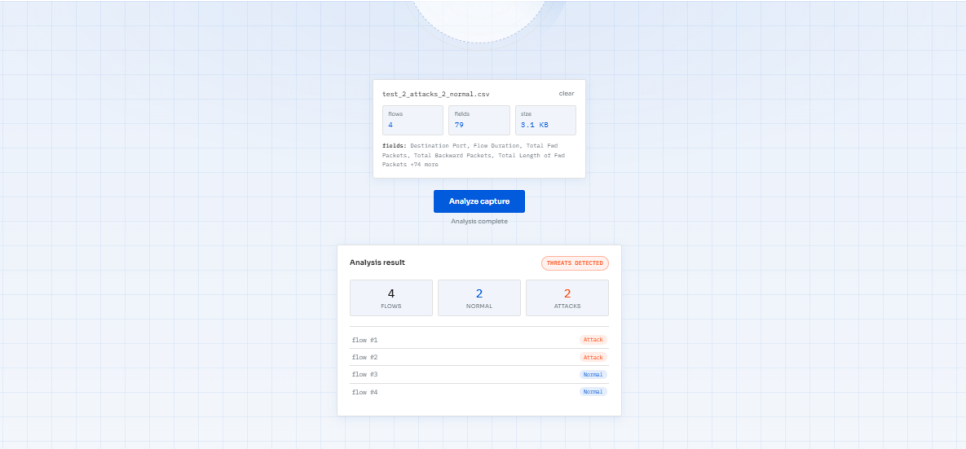


Figure 4.14 : Résultat de la prédiction après import CSV

Conclusion

Dans ce chapitre, nous avons présenté les différentes interfaces de notre application de détection d'intrusions, ainsi que les technologies utilisées pour son développement. Ces interfaces, simples et intuitives, permettent à l'utilisateur d'interagir facilement avec le modèle de Machine Learning développé, tout en assurant une expérience fluide et sécurisée. Ce chapitre marque ainsi la fin de la présentation de la réalisation pratique de notre projet.

Conclusion générale

Dans un contexte marqué par l'augmentation constante des cybermenaces, ce projet nous a permis de concevoir et de développer un système de détection d'intrusions basé sur l'apprentissage automatique. L'objectif principal était de proposer une solution capable de distinguer efficacement le trafic réseau normal du trafic malveillant, tout en offrant une interface simple et accessible aux utilisateurs.

Pour répondre à cette problématique, nous avons exploité le jeu de données CICIDS2017, largement reconnu dans le domaine de la cybersécurité, que nous avons analysé, nettoyé et préparé avant de l'utiliser pour l'entraînement de plusieurs modèles de classification. Après comparaison des performances du Decision Tree et du Random Forest à l'aide de métriques telles que l'Accuracy, la Precision, le Recall et le F1-Score, le modèle Random Forest s'est révélé être le plus performant, avec une Accuracy de 99,93% et un F1-Score de 97,58%. Ce modèle a ensuite été intégré au sein d'une application web développée avec Flask, permettant à l'utilisateur d'analyser le trafic réseau soit par saisie manuelle des caractéristiques, soit par importation d'un fichier CSV, et d'obtenir instantanément le résultat de la classification.

Sur le plan des objectifs fixés au début de ce projet, l'ensemble des fonctionnalités prévues a pu être réalisé : la préparation rigoureuse des données, l'entraînement et la comparaison de plusieurs modèles de Machine Learning, ainsi que le développement d'une application web fonctionnelle intégrant le modèle retenu. Ce travail a ainsi permis de mettre en pratique, de manière concrète, les concepts théoriques abordés durant notre formation.

Sur le plan personnel, la réalisation de ce projet a été une expérience particulièrement enrichissante. Elle m'a permis d'approfondir mes connaissances en Machine Learning, de me familiariser avec les problématiques réelles liées à la cybersécurité, et de renforcer mes compétences en développement web. Elle m'a également appris à gérer un projet de bout en bout, depuis l'analyse du besoin jusqu'à la réalisation d'une solution complète et fonctionnelle, en surmontant les différentes difficultés techniques rencontrées tout au long du parcours.

Ce travail présente toutefois certaines limites. Le modèle a été entraîné sur un sous-ensemble du dataset CICIDS2017, ce qui peut restreindre sa capacité de généralisation face à des types d'attaques non représentés dans les données d'entraînement. De plus, le système actuel repose sur une analyse a posteriori du trafic (saisie manuelle ou fichier CSV), et ne permet pas encore une détection en temps réel directement sur le flux réseau.

Plusieurs perspectives peuvent ainsi être envisagées pour prolonger ce travail. Il serait notamment intéressant d'intégrer une capture et une analyse du trafic réseau en temps réel, d'étendre l'entraînement

à des jeux de données plus diversifiés afin d'améliorer la robustesse du modèle face à de nouvelles formes d'attaques, ou encore d'explorer des architectures plus avancées telles que les réseaux de neurones profonds. Ces perspectives ouvrent la voie à une solution encore plus complète et adaptée aux exigences de la cybersécurité moderne.

Bibliographie

[1] A. L. Buczak et E. Guven, « A Survey of Data Mining and Machine Learning Methods for Cyber Security Intrusion Detection, » *IEEE Communications Surveys & Tutorials*, t. 18, n° 2, p. 1153-1176, 2016.

[2] K. Scarfone et P. Mell, « Guide to Intrusion Detection and Prevention Systems (IDPS), » National Institute of Standards et Technology, rapp. tech. NIST Special Publication 800-94, 2007.

[3] I. Sharafaldin, A. H. Lashkari et A. A. Ghorbani, « Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization, » in *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, 2018, p. 108-116.

[4] J. R. Quinlan, « Induction of Decision Trees, » *Machine Learning*, t. 1, n° 1, p. 81-106, 1986.

[5] L. Breiman, « Random Forests, » *Machine Learning*, t. 45, n° 1, p. 5-32, 2001.

[6] Pallets Projects. « Flask Documentation, » visité le 15 sept. 2026. adresse : `https://flask.palletsprojects.com/`

[7] F. Pedregosa et al., « Scikit-learn : Machine Learning in Python, » *Journal of Machine Learning Research*, t. 12, p. 2825-2830, 2011.

[8] Canadian Institute for Cybersecurity. « CICIDS2017 Dataset, » visité le sept. 2026. adresse : `https://www.unb.ca/cic/datasets/ids-2017.html`

Annexes

Annexe 1. Liste complète des caractéristiques utilisées

Le tableau suivant présente l'ensemble des 23 caractéristiques utilisées pour l'entraînement du modèle de classification.

Caractéristique	**Description**
Flow Duration	Durée du flux réseau
Total Length of Fwd Packets	Longueur totale des paquets transmis
Fwd Packet Length Max	Taille maximale des paquets transmis
Fwd Packet Length Mean	Taille moyenne des paquets transmis
Fwd Packet Length Std	Écart-type de la taille des paquets transmis
Flow Bytes/s	Nombre d'octets transmis par seconde
Flow Packets/s	Nombre de paquets transmis par seconde
Flow IAT Std	Écart-type de l'intervalle entre paquets
Flow IAT Max	Intervalle maximal entre paquets
Fwd IAT Max	Intervalle maximal entre paquets sortants
Fwd Header Length	Longueur de l'en-tête des paquets sortants
Fwd Packets/s	Nombre de paquets sortants par seconde
Bwd Packets/s	Nombre de paquets entrants par seconde
Max Packet Length	Longueur maximale des paquets
Packet Length Mean	Longueur moyenne des paquets
Packet Length Std	Écart-type de la longueur des paquets
Packet Length Variance	Variance de la longueur des paquets
Average Packet Size	Taille moyenne des paquets
Avg Fwd Segment Size	Taille moyenne des segments sortants
Fwd Header Length.1	Longueur de l'en-tête (variante)
Subflow Fwd Bytes	Nombre d'octets sortants par sous-flux
Init_Win_bytes_forward	Taille initiale de la fenêtre TCP sortante
Init_Win_bytes_backward	Taille initiale de la fenêtre TCP entrante

Tableau 4.1 : Liste des 23 caractéristiques utilisées pour l'entraînement

Annexe 2. Guide d'installation et d'exécution

Prérequis

- Python 3.9 ou version supérieure
- pip (gestionnaire de paquets Python)

Installation

Cloner ou télécharger le projet.

Installer les dépendances nécessaires :

```
pip install -r requirements.txt
```

Lancer l'application :

```
python app.py
```

Ouvrir un navigateur et accéder à l'adresse :

```
http://127.0.0.1:5000
```

Résumé

Ce projet a pour objectif la conception et le développement d'un système de détection d'intrusions réseau basé sur des techniques de Machine Learning. Face à l'augmentation constante des cybermenaces, notre solution exploite le jeu de données CICIDS2017 afin d'entraîner et de comparer plusieurs modèles de classification, notamment le Decision Tree et le Random Forest. Après évaluation à l'aide de métriques telles que l'Accuracy, la Precision, le Recall et le F1-Score, le modèle Random Forest a été retenu comme solution optimale, atteignant une Accuracy de 99,93%. Ce modèle a ensuite été intégré au sein d'une application web développée avec Flask, permettant d'analyser le trafic réseau par saisie manuelle ou par importation d'un fichier CSV, et d'obtenir instantanément le résultat de la classification (trafic normal ou attaque).

Mots clés : Détection d'intrusions, Machine Learning, Random Forest, Cybersécurité, CICIDS2017, Flask.

Abstract

This project focuses on the design and development of a network intrusion detection system based on Machine Learning techniques. Given the constant rise of cyberthreats, our solution leverages the CICIDS2017 dataset to train and compare several classification models, namely Decision Tree and Random Forest. After evaluation using metrics such as Accuracy, Precision, Recall and F1-Score, the Random Forest model was selected as the optimal solution, achieving an Accuracy of 99.93%. This model was then integrated into a web application developed with Flask, allowing users to analyze network traffic either through manual input or CSV file upload, and to instantly obtain the classification result (normal traffic or attack).

Keywords : Intrusion Detection, Machine Learning, Random Forest, Cybersecurity, CICIDS2017, Flask.